

## Self-stabilizing Algorithms for Orderings and Colorings

WAYNE GODDARD, STEPHEN T. HEDETNIEMI

DAVID P. JACOBS, PRADIP K. SRIMANI  
*Department of Computer Science, Clemson University*  
*Clemson, SC 29634-0974 USA*

Received (received date)

Revised (revised date)

Communicated by Editor's name

### ABSTRACT

A  $k$ -forward numbering of a graph is a labeling of the nodes with integers such that each node has less than  $k$  neighbors whose labels are equal or larger. Distributed algorithms that reach a legitimate state, starting from any illegitimate state, are called *self-stabilizing*. We obtain three self-stabilizing (s-s) algorithms for finding a  $k$ -forward numbering, provided one exists. One such algorithm also finds the  $k$ -height numbering of graph, generalizing s-s algorithms by Bruell et al. [4] and Antonoiu et al. [1] for finding the center of a tree. Another  $k$ -forward numbering algorithm runs in polynomial time. The motivation of  $k$ -forward numberings is to obtain new s-s graph coloring algorithms. We use a  $k$ -forward numbering algorithm to obtain an s-s algorithm that is more general than previous coloring algorithms in the literature, and which  $k$ -colors any graph having a  $k$ -forward numbering. Special cases of the algorithm 6-color planar graphs, thus generalizing an s-s algorithm by Ghosh and Karaata [13], as well as 2-color trees and 3-color series-parallel graphs. We discuss how our s-s algorithms can be extended to the synchronous model.

*Keywords:* self-stabilizing algorithms, graph, coloring,  $k$ -forward numbering.

## 1. Introduction

One of the most important requirements of modern distributed systems is robustness. A goal of a distributed system is that the system should function correctly in spite of intermittent faults. Ideally the global state of the system should remain in a legitimate state. Often, however, malfunctions or perturbations bring the system to some illegitimate state, and it is desirable that the system be automatically brought back to a legitimate state without the interference of an external agent.

The traditional approach to fault tolerance uses fault masking and is *pessimistic* in the sense that it assumes the worst case scenario and protects the system against such an eventuality; validity is guaranteed in the presence of faulty processes, which necessitates restrictions on the number of faults and on the fault model. Fault masking is not free; it requires additional hardware or software, and it considerably in-

creases the cost of the system. This additional cost may not be an economic option, especially when most faults are transient in nature and a temporary unavailability of a system service is acceptable for a short period of time.

Systems that reach a legitimate state starting from any illegitimate state in a finite number of steps are called *self-stabilizing systems* [7, 8]. This kind of property is highly desirable for any distributed system, since without having a global memory, global synchronization is achieved in finite time and thus the system can correct itself automatically from spurious perturbations or failures.

Self-stabilization [10] is an *optimistic* way of looking at system fault tolerance, because it provides a built-in safeguard against transient failures that might corrupt the data in a distributed system. The objective of self-stabilization (as opposed to masking faults) is to recover from failure in a reasonable amount of time and without intervention by any external agent. Since the faults are transient (eventual repair is assumed), it is no longer necessary to assume a bound on the number of failures. Self-stabilization has potential uses in ad-hoc networks [11], sensor networks [18], and peer-to-peer communication [6].

We assume the state-reading model in which each node  $v$  can read the variables of its neighbors  $N(v)$ , and can make decisions based on the contents of its neighbors' local variables. The contents of a node's local variables determine its *local state*. A node may change its local state by making a *move*, that is, changing the values of its local variables. Self-stabilizing algorithms are given as a set of rules of the form **if**  $p(v)$  **then**  $M$ , where  $p(v)$  is a predicate and  $M$  is a move. A node  $v$  becomes *privileged* if at least one of its predicates  $p(v)$  is true. When a node becomes privileged, it may execute the corresponding move (i.e., change its local variables). When no nodes are privileged, we say that the system is *stable*.

Unless otherwise specified, all algorithms in this paper are *self-stabilizing* or *s-s algorithms*. For most of this paper we assume that there exists a (not necessarily fair) *central daemon* which selects one of the privileged nodes to move, such that no two nodes move at the same time. Also, we present our algorithms for an *anonymous* network where nodes do not have ID's. In Section 5, however, we consider how our s-s algorithms are effected under other distributed models, namely the so-called *synchronous* distributed model and the *distributed daemon* model.

## 2. Overview of results

Although the goal of this paper is to obtain coloring algorithms, it will be necessary to first study *k-forward numberings*. For a graph  $G = (V, E)$ , we say that a function

$$c : V \rightarrow \mathbb{N}$$

which assigns to the nodes, not necessarily distinct, non-negative integers, is a *k-forward numbering* if each node has less than  $k$  neighbors which are assigned equal or larger numbers. That is, for any node  $v$ ,

$$|\{u \in N(v) : c(u) \geq c(v)\}| < k.$$

It will be convenient to define a *k-forward ordering* as a total ordering  $v_1, \dots, v_n$  that is a *k-forward numbering*. That is,  $|\{v_j \in N(v_i) : j > i\}| < k$ , for any  $i$ . Throughout this paper, we assume  $k \geq 2$  is a fixed integer.

Note that every graph has a *k-forward ordering* for sufficiently large  $k$ ; if  $G$  has maximum degree  $\Delta$ , then trivially, by letting  $k = \Delta + 1$ , any total ordering is a *k-forward ordering*. As is often the case in algorithmic design, one restricts the network topology to a special class of graphs such as rings or trees. We consider the class of graphs having *k-forward orderings*. All of our algorithms, for a fixed  $k$ , have the unusual property that they stabilize if and only if the graph has a *k-forward ordering*.

Consider some examples of *k-forward orderings*. First, any tree has a 2-forward ordering because, after rooting the tree, the nodes can be ordered bottom-up beginning with leaves, assigning the largest number to the root. No cycle has a 2-forward ordering because the node with minimum value would have two neighbors with greater values. It follows that a graph  $G$  has a 2-forward ordering if and only if  $G$  is a forest. Because cycles have maximum degree 2, any total order is a 3-forward ordering, but note also that a unicyclic graph can be 3-forward ordered by numbering leaves first, and working in toward the cycle. It is well-known that all planar graphs have 6-forward orderings. But Figure 1 shows a planar graph having a 4-forward ordering, and it is easy to show that this graph has no 3-forward ordering. Finally, in Section 3.1 we'll examine a class of nonplanar graphs that have 3-forward orderings.

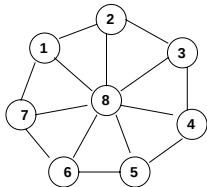


Fig. 1. graph with a 4-forward ordering

The initial focus of this paper is s-s algorithms for finding *k-forward numberings*. Three such algorithms are given. On the surface, the algorithms appear simple in that each uses only a single nonnegative integer variable and has a single rule for comparing its value with the value of its neighbors. However, their analyses and correctness proofs are not simple. But what is most surprising is that the subtle differences in the three algorithms produce vastly different behaviors and running times.

In Section 3.1 we give the simplest of our algorithms, which tries to resolve local conflicts by always increasing a local variable to be larger than its neighbors. The algorithm finds a *k-forward numbering* if and only if one exists, but we show, by example, that the number of moves can be exponential. In Section 3.2 we give an s-s algorithm for finding a special kind of *k-forward numbering*, called a *k-height numbering*. This algorithm generalizes the algorithms of Bruell et al. [4] and Antonoiu et al. [1] for finding the center of a tree. In Section 3.3 we give

a  $k$ -forward numbering algorithm that stabilizes in time that is polynomial in  $n$ , the network size, and  $W$ , the largest number at initialization. All three of these numbering algorithms find 2-forward numberings on trees, 3-forward numberings on series-parallel graphs, 6-forward numberings on planar graphs, and  $k$ -forward numberings on partial  $(k - 1)$ -trees.

The ultimate goal of this paper is to obtain new s-s coloring algorithms. A *coloring* of a graph  $G$  is an assignment of integers (or colors) to the nodes such that no two adjacent nodes receive the same color. The *chromatic number* of a graph  $G$  is the minimum number of colors used in a coloring, and is denoted  $\chi(G)$ . Unlike  $\chi(G)$ , we can compute  $col(G)$  in polynomial time. A well-known theorem of Brooks [17] says that any for any graph  $G$ ,  $\chi(G) \leq \Delta(G) + 1$ , where  $\Delta(G)$  denotes the maximum degree of a node in  $G$ . Self-stabilizing algorithms which find  $(\Delta + 1)$ -colorings, under various models, are presented by Dolev and Herman [9], Gradinariu and Tixeuil [14], and Hedetniemi et al. [16]. Ghosh and Karaata in [13] give an s-s algorithm to find a 6-coloring of a planar graph, using the fact that every planar graph has a node of degree less than 6.

The *coloring number* of a graph  $G$ , denoted  $col(G)$ , is the smallest number  $k$  such that  $G$  has a  $k$ -forward numbering. Since any  $k$ -forward numbering can be used to greedily  $k$ -color a graph [17], it follows that  $\chi(G) \leq col(G) \leq \Delta(G) + 1$ , which is a strengthening of Brooks' Theorem. In a *list coloring*, each node is assigned a list of available colors, and the algorithm must produce a coloring, using a color from each node's list. In Section 4.1 we give a linear-time s-s list-coloring algorithm that assumes a  $k$ -forward numbering is given.

In Section 4.2 we give an s-s coloring algorithm that will  $k$ -color any graph having a  $k$ -forward numbering. Special cases of this algorithm 6-color any planar graph, thus generalizing an s-s algorithm of Ghosh and Karaata [13], 2-color any tree and 3-color any series-parallel graph.

In Section 5 we show that our algorithms self-stabilize even with the synchronous (maximal parallelism) model, thus generalizing results of Shukla et al. [19].

Finally, in Section 6 we show how to calculate for  $k$ -trees what is called the *kernel*, a consequence of which is that dynamic programming algorithms on these graphs can be made self-stabilizing. This generalizes results for trees given by Ghosh et al. [12]. Indeed, our results improve by a factor of  $n$  the dynamic programming algorithms of that paper. Their results have also been improved by Blair and Manne in [3], but ours is better in terms of storage.

### 3. $k$ -forward number

In this section, we examine three algorithms for finding  $k$ -forward numberings. All three algorithms are simple, in that each involves a single rule and a single integer variable. The first algorithm, which we call **Leap**, is the natural generalization of an algorithm by Ghosh and Karaata [13]. However, we show that, while it is efficient for  $k = 2$ , their algorithm can take exponentially many moves. The second algorithm, which we call **Slide**, is a generalization of center-finding algorithms by Bruell et al. [4] and Antonoiu et al. [1]. This algorithm has the advantage of always

|                                                                                                                                                         |
|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>variable:</b> nonnegative integer $w(v)$<br><b>leap:</b> if $ \{u \in N(v) : w(u) \geq w(v)\}  \geq k$<br>set $w(v) = 1 + \max\{w(u) : u \in N(v)\}$ |
|---------------------------------------------------------------------------------------------------------------------------------------------------------|

Fig. 2. Algorithm **Leap**

producing a unique numbering, called a  $k$ -height numbering, which identifies a generalization of centers in other graphs. However the complexity of Algorithm **Slide** remains undetermined. The third algorithm, which we call **UpSlide**, takes a new strategy. We show that it has a guaranteed quadratic performance. The following easy observation will be useful.

**Lemma 1** *A graph has a  $k$ -forward ordering if and only if it has a  $k$ -forward numbering.*

**Proof.** By definition,  $k$ -forward orderings are  $k$ -forward numberings. Conversely, if  $w : V \rightarrow \mathbb{N}$  is a  $k$ -forward numbering, then we may order the vertices so that

$$w(v_1) \leq w(v_2) \leq \dots \leq w(v_n).$$

It is easy to see that the total order  $v_1, \dots, v_n$  is a  $k$ -forward ordering.  $\square$

### 3.1. $k$ -forward numbering: **Leap**

In this section we present Algorithm **Leap**, shown in Figure 2, an s-s algorithm for finding a  $k$ -forward numbering. Each node  $v$  maintains a single integer variable  $w(v)$ . The algorithm has a single rule: if a node  $v$  has at least  $k$  neighbors having the same number or larger number, it sets  $w(v)$  to be one larger than the maximum number of a neighbor. This is the natural generalization an s-s algorithm of Ghosh and Karaata [13], while avoiding the use of IDs. The fact that IDs are unnecessary was observed by Shukla et al. in [19].

Let  $G = (V, E)$  be a graph, and let  $(v_1, \dots, v_n)$  be any fixed, total ordering of  $V$ . Assuming that edges  $e = v_i v_j$ , are always written with  $i < j$ , this induces a total ordering  $(E, <)$  on  $E$  where, if  $e = v_i v_j$  and  $e' = v_{i'} v_{j'}$ , then  $e < e'$  if and only if either  $i < i'$ , or  $i = i'$  and  $j < j'$ . Finally, we obtain a total order on the powerset  $2^E$  in the usual lexicographic way: given  $S, S' \subseteq E$ ,  $S \neq S'$ , if  $e$  is the smallest edge in  $(S - S') \cup (S' - S)$ , then  $S < S'$  if and only if  $e \in S'$ . Note that  $E$  is the maximum element under this total order.

Now assume that  $(v_1, \dots, v_n)$  is a *fixed*  $k$ -forward ordering of  $G$ . Let  $w : V \rightarrow \mathbb{N}$  be an arbitrary numbering of  $V$ . An edge  $e = v_i v_j$ , where  $i < j$ , is called a *bad* edge, relative to  $w$ , if  $w(v_i) \geq w(v_j)$ , and a *good* edge otherwise.

**Lemma 2** *Assume that  $G$  has a  $k$ -forward ordering  $(v_1, \dots, v_n)$ . In Algorithm **Leap**, let  $B_t$  be the set of bad edges relative to this ordering at time  $t$ . Then  $B_{t+1} < B_t$ , in the total order on  $2^E$ .*

**Proof.** Assume that node  $v = v_i$  moves at time  $t$ . Before the move, let

$$S = \{u \in N(v) \mid w(v) \leq w(u)\}.$$

Since  $v$  was privileged,  $|S| \geq k$ . Let  $S = \{v_{j_1}, \dots, v_{j_s}\}$ , and without loss of generality, we may assume

$$j_1 < j_2 < \dots < j_s.$$

Note that  $j_1 < i$ , for otherwise we would not have a  $k$ -forward ordering. Therefore, the move causes the edge  $v_{j_1}v_i$  to change from bad to good. Let  $A = \{vu \mid u \in S\}$ , and  $A' = \{vu \mid u \in N(v) - S\}$ . We have observed that the minimum member of  $A$  changes from bad to good. The status of each edge in  $A'$ , however, remains unchanged, as does the status of all edges not incident with  $v$ . Therefore  $B_{t+1} < B_t$ , completing the proof.  $\square$

**Theorem 1** *Algorithm Leap stabilizes if and only if the graph has a  $k$ -forward numbering. If it stabilizes, it produces a  $k$ -forward numbering.*

**Proof.** If Algorithm Leap stabilizes, then no node is privileged, so the numbers  $w(v)$  form a  $k$ -forward numbering. On the other hand, assume there is a  $k$ -forward numbering. By Lemma 1, there is a  $k$ -forward ordering  $(v_1, \dots, v_n)$ . By Lemma 2 the algorithm can execute only finitely many moves, since each move causes the bad edge sets  $B_t$  to decrease under their total order.  $\square$

**Quadratic for  $k = 2$ :** We now observe that on trees, Algorithm Leap takes only  $O(n^2)$  moves, when  $k = 2$ . Let  $T$  be a tree, and let  $v_1, v_2, \dots, v_n$  be a 2-forward ordering of its nodes, where  $v_n$  is the root, and each node has a greater index than all of its children. As before, define an edge  $v_i v_j$ ,  $i < j$ , to be bad if  $w(i) \geq w(j)$ , and good otherwise. Let  $B$  be the set of bad edges. If  $v_i$  moves, then there is at least one bad edge, incident to one of its children, that becomes good. If  $v_i$  is not the root, after the move, there is exactly one bad edge incident to  $v_i$ , namely the edge to its parent. If  $v_i$  is the root, then after the move there are no bad edges incident to  $v_i$ . In either case, the sum

$$\Phi = \sum_{\substack{v_i v_j \in B \\ i < j}} (n - j)$$

must decrease.

**Exponential for  $k \geq 3$ :** We now show that Algorithm Leap can take exponential time. Set  $k = 3$ . We let  $H_n$  be the graph shown in Figure 3, having  $n$  pairs of

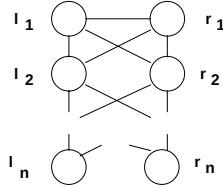


Fig. 3. the graph  $H_n$

nodes, arranged in rows. Nodes in row  $i$ ,  $1 \leq i \leq n - 1$ , are adjacent to both nodes in row  $i + 1$ . Moreover, in row one, the nodes  $l_1$  and  $r_1$  are adjacent.

We assign numbers to the nodes, which represent the values  $w(v)$  from Algorithm **Leap**. A numbering of  $H_n$  is said to be in *final form* if both numbers in row  $i$  are greater than both numbers in row  $i + 1$ . Such a numbering is a 3-forward numbering. Next, we say that a numbering of  $H_n$  is in *initial form* if

1. Each number in row two is greater than both numbers in row one.
2. In rows  $i \geq 3$ , some number is greater than both numbers in row  $i - 1$ .

Figure 4 shows  $H_3$  in both initial and final form.

The following technical lemma will simplify the subsequent theorem.

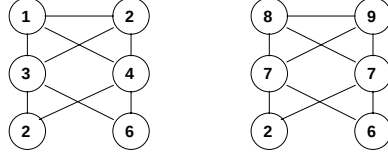


Fig. 4. An initial form and final form of  $H_3$

**Lemma 3** Assume in  $H_n$ ,  $n \geq 3$ , that for each  $i \leq n - 2$ , both numbers in row  $i$  are greater than the numbers in row  $i + 1$ , and suppose also that node  $l_{n-1}$  is privileged. Then there is a sequence of  $2(n - 2)$  moves such that the subgraph consisting of the first  $n - 1$  rows is initial form.

**Proof.** It is easy to check that the following sequence of moves by nodes

$$l_{n-1}, l_{n-2}, \dots, l_2, r_2, \dots, r_{n-2}, r_{n-1}$$

give the desired result. □

We now define the function

$$t_n = \begin{cases} 2 & \text{if } n = 2 \\ 2t_{n-1} + 2(n - 2) & \text{if } n \geq 3 \end{cases}$$

**Theorem 2** If  $H_n$  is in initial form, there is a sequence of  $t_n$  moves that terminate in final form.

**Proof.** By induction on  $n$ . When  $n = 2$  it is clear that  $t_2 = 2$  moves transform an initial form into a final form. When  $n = 3$ , it is easy to see that any initial form may be transformed into a final form in  $t_3 = 6$  moves. Now let  $n \geq 4$  and assume the result for all initial forms of  $H_{n-1}$ . We consider the graph  $H_n$  in the left of Figure 5, assumed to be in initial form. To complete the induction, it suffices to show there is a sequence of  $t_n$  moves that terminate in a final form.

Let  $H_{n-1}$  be the subgraph consisting of the first  $n - 1$  rows. Then  $H_{n-1}$  is also in initial form. By our induction assumption, there is a sequence of  $t_{n-1}$  moves taking  $H_{n-1}$  into final form, as shown in the right of Figure 5.

Note that the numbers  $a_{n-1}$  and  $b_{n-1}$  remain unchanged, since they have degree two in  $H_{n-1}$ . Since both values in row  $n - 2$  are greater than those in row  $n - 1$ , and since one of the values in row  $n$  is greater than both in row  $n - 1$ , it follows

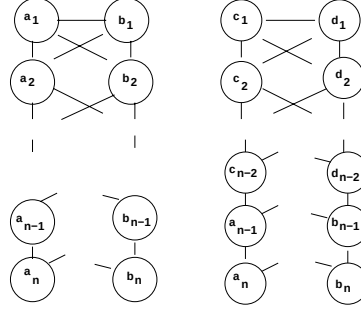


Fig. 5.  $H_{n-1}$  transformed to final form.

that both nodes  $l_{n-1}$  and  $r_{n-1}$  are privileged. By Lemma 3, there is a sequence of  $2(n-2)$  moves that places  $H_{n-1}$  in initial form again. Applying the induction assumption again, in  $t_{n-1}$  moves,  $H_{n-1}$  is transformed into a final form. This places  $H_n$  in final form as well, since the numbers in row  $n-1$  now are greater than those in row  $n$ . Since  $t_n = t_{n-1} + 2(n-2) + t_{n-1}$ , we are done.  $\square$

For each  $k \geq 4$ , it is also possible to construct a similar family of graphs which allow exponential running times. This shows that the coloring algorithm in [13] runs in exponential time.

### 3.2. $k$ -height numbering: Slide

In this section we present a variation of Algorithm **Leap**. It finds a special kind of  $k$ -forward numbering by taking a more conservative approach to adjusting values. This time, if a node has too many larger neighbors, it adjusts its value to be just large enough so that the desired  $k$ -forward property holds locally. If the algorithm stabilizes, it results in a *unique* numbering, regardless of the initial conditions.

The resulting algorithm is the natural generalization of existing s-s algorithms for finding centers. Recall that the *eccentricity* of a node is the maximum length of a shortest path to some other node, and the *center* of a graph is the set of nodes having smallest eccentricity. There are either one or two central nodes in a tree. The center of a tree can be found by first numbering the leaves with zero, and then at stage  $i \geq 1$ , numbering with  $i$  the leaves in the subgraph of unnumbered nodes. The center of a tree consists of the node or two nodes with greatest number, when all nodes have been numbered.

This numbering can be generalized in the following way: first, number with zero, all nodes having degree less than  $k$ . Then at stage  $i \geq 1$ , number with  $i$  those nodes having degree less than  $k$  in the subgraph of unnumbered nodes. Such a numbering will be called the  $k$ -height numbering of  $G$ . The  $k$ -height of a node  $v$  is the number thus assigned, denoted  $h_k(v)$ , and the  $k$ -height of  $G$ , denoted  $h_k(G)$ , is the largest such number. When  $k$  is clear from the context, we sometimes will refer to simply the *height* of  $v$ .

Note that when the graph is a tree, and  $k = 2$ , this amounts to the center finding algorithm described above. Hence, all trees can be 2-height numbered. For a given

**variable:** nonnegative integer  $w(v)$   
**slide:** if  $w(v) \neq x(v)$   
           set  $w(v) = x(v)$

Fig. 6. Algorithm Slide

$k$ , an arbitrary graph  $G$  might not have a  $k$ -height numbering, but if  $k$  is sufficiently large,  $G$  will have a  $k$ -height numbering.

**Lemma 4** *A graph has a  $k$ -height numbering if and only if it has a  $k$ -forward numbering.*

**Proof.** By the construction, any  $k$ -height numbering is a  $k$ -forward numbering. Conversely, assume that  $G$  has a  $k$ -forward numbering  $w : V \rightarrow \mathbb{N}$ . Since  $w$  is a  $k$ -forward numbering, the nodes with smallest  $w$  value must have degree less than  $k$ . Letting  $S_0$  denote the set of nodes having degree less than  $k$ , we number all nodes in  $S_0$  with 0. If  $V - S_0 \neq \emptyset$ , let  $S_1$  be the subset of nodes of  $V - S_0$  that are adjacent to less than  $k$  members of  $V - S_0$ . Such a node must exist; in particular, the unnumbered node having smallest  $w$  value satisfies this property. We assign the value 1 to members of  $S_1$ , and continue. Eventually we achieve a  $k$ -height numbering.  $\square$

Algorithm Slide, shown in Figure 6, is an algorithm for finding a  $k$ -height numbering of a graph, if one exists. This algorithm extends the center-finding algorithms for trees [1, 4], to graphs having coloring number  $k$ .

For a multiset of integers  $S = \{s_i\}_{i=1}^r$  where

$$s_1 \geq s_2 \geq \dots \geq s_r$$

define  $r_k(S) = s_k$ , the  $k$ th largest value in  $S$ . Each node  $v$  in Algorithm Slide maintains a nonnegative variable  $w(v)$ . For a node  $v$ , we define the *consistent number*  $x(v)$  by

$$x(v) = \begin{cases} 0 & \text{if } v \text{ has less than } k \text{ neighbors} \\ 1 + r_k(\{w(u) : u \in N(v)\}) & \text{otherwise.} \end{cases}$$

Note that the consistent number of a node  $v$  is precisely the smallest nonnegative number for which  $v$  will have fewer than  $k$  neighbors whose values are greater or equal.

**Lemma 5** *Let  $G$  be a graph whose  $k$ -height is  $h$ . During Algorithm Slide, the values  $w(v)$  can never exceed  $W + h$ , where  $W$  is the largest initial value of  $w(v)$ .*

**Proof.** By induction on  $h$ , we show that, for nodes  $v$  having  $k$ -height  $h$ , the values  $w(v)$  can never exceed  $W + h$ . First, let  $v$  be a node of height  $h = 0$ . If node  $v$  never moves, then obviously the condition  $w(v) \leq W$  must always exist. But if node  $v$  makes a first move, then it becomes  $x(v) = 0$  permanently, since it has fewer than  $k$  neighbors. By induction, now assume that for all nodes  $u$  having height  $h' < h$ , the condition  $w(u) \leq W + h'$  is maintained during any execution. Now let  $v$  be a node that moves, having height  $h$ . Consider its neighborhood  $N(v)$ : less than  $k$  of

the members of  $N(v)$  have height  $h$  or greater. So among the  $k$  neighbors having largest value, at least one, say  $u$ , has height  $h' < h$ . So we have

$$x(v) \leq w(u) + 1 \leq W + h' + 1 \leq W + h,$$

the middle inequality following from our induction assumption.  $\square$

In Algorithm Slide a move by a node  $v$  always changes the value of  $w(v)$ , so we may speak of a move as either being *increasing* or *decreasing*. For a node  $v$  and a natural number  $r$ , let  $d_v(r)$  denote a decreasing move by a node  $v$  to  $r$ . Let  $D_v(r)$  be the number of decreasing moves made by  $v$  to  $r$ . We omit the proof of the following lemma, which is trivial.

**Lemma 6** *Let  $S$  be a multiset of integers and let  $S' = S - \{s_i\} \cup \{s'_i\}$  for some  $s_i \in S$ . If  $r_k(S') < r_k(S)$  then  $s'_i \leq r_k(S')$ .*

**Lemma 7** *Let  $M_{t_1}$  and  $M_{t_2}$  be moves made by the same node  $v$  at times  $t_1 < t_2$ , and where  $M_{t_2} = d_v(r)$ . Then at some time in the interval  $(t_1, t_2)$  there exists a decreasing move  $d_u(s)$  by a node  $u$  adjacent to  $v$  with  $s < r$ .*

**Proof.** We may assume  $v$  makes no moves in the interval  $(t_1, t_2)$ . Let  $S_t$  denote the multiset of numbers  $\{w(u) : u \in N(v)\}$  at time  $t$ . Because node  $v$  moves at time  $t_1$ , at time  $t_1 + 1$ ,  $w(v) = x(v) = r_k(S_{t_1+1}) + 1$ . At time  $t_2$ , however,  $v$  makes a decreasing move to  $w(v) = x(v) = r_k(S_{t_2}) + 1 = r$ . This means that in the time interval  $(t_1, t_2)$ ,  $r_k$  decreased to  $r - 1$ . By Lemma 6 it follows that there was a move  $d_u(s)$  for  $s \leq r - 1$ .  $\square$

**Lemma 8** *For any natural number  $r$ , the sum  $\sum_{v \in V} D_v(r)$  is finite.*

**Proof.** We use mathematical induction on  $r$ . In the basis case, when  $r = 0$ , we see that each node  $v$  can make at most one move  $d_v(0)$ . By our induction hypothesis, we may assume that  $D_u(s)$  is finite for all  $u$  and  $s < r$ . Now suppose by contradiction that  $D_v(r)$  is not finite. By Lemma 7, between any two consecutive moves  $d_v(r)$ , is a decreasing move  $d_u(s)$ , where  $s < r$  by a neighbor  $u$ . Thus for some *particular*  $u$  and  $s < r$  there must be infinitely many moves  $d_u(s)$ , contradicting the induction hypothesis.  $\square$

Bruell et al. [4] show that for  $k = 2$  there can be at most  $O(n^2)$  decreasing moves. However, that result exploits the fact that in trees there is a unique path between any two nodes. There does not appear to be an equivalent result for  $k \geq 3$ .

**Theorem 3** *Algorithm Slide stabilizes on a graph  $G$  if and only if  $G$  has a  $k$ -forward numbering.*

**Proof.** It is easy to see that if Algorithm Slide stabilizes, then each node  $v$  has fewer than  $k$  neighbors of greater or equal value, and so the numbers  $w(v)$  form a  $k$ -forward numbering. Conversely, assume  $G$  has a  $k$ -forward numbering. By Lemma 4, it has a  $k$ -height numbering. Then by Lemma 5, the values  $w(v)$  are bounded by  $B = W + n$ . Therefore, the total number of decreasing moves is exactly

$$D = \sum_{0 \leq r < B} \sum_v D_v(r),$$

**variable:** nonnegative integer  $w(v)$   
**upslide:** if  $v$  has at least  $k$  larger or equal neighbors  
           set  $w(v) = 1 + r_k(\{w(u) : u \in N(v)\})$

Fig. 7. Algorithm UpSlide

which by Lemma 8, is finite. The number of increasing moves must therefore be finite since a node can make no more than  $B$  increasing moves before making a decreasing move. The number of moves is therefore finite, completing the proof.  $\square$

**Theorem 4** *At stabilization, the values  $w(v)$  of Algorithm Slide are a  $k$ -height numbering.*

**Proof.** When Algorithm Slide stabilizes, Theorem 3 and Lemma 4 together imply that  $G$  has a  $k$ -height numbering. We will argue, using induction on  $h$ , that at stabilization,  $w(v) = h$  if and only if  $v$  has height  $h$ . It is easy to see this holds for  $h = 0$ , so suppose this condition holds for all heights less than  $h$ . To complete the induction we must show

- (i) if  $h(v) = h$  then  $w(v) = h$ ,
- (ii) if  $w(v) = h$  then  $h(v) = h$ .

First, suppose that  $h(v) = h$ . Let  $N(v) = A \cup B \cup C$ , where  $A$  contains the neighbors of height at least  $h$ ,  $B$  contains the neighbors of height  $h - 1$ , and  $C$  contains the remaining neighbors. If  $a = |A|$  and  $b = |B|$ , it follows that  $a + b \geq k$ , for otherwise  $v$  would have smaller height. Also, by definition of height,  $a < k$ , and so  $b > 0$ . By the induction assumption,  $w(u) \geq h$  for each  $u \in A$ ,  $w(u) = h - 1$  for each  $u \in B$ , and  $w(u) < h - 1$  for each  $u \in C$ . Thus  $x(v) = h$ , and so  $w(v) = h$ , completing part (i) of the induction.

To establish part (ii), assume that  $h(v) = h' \neq h$ . We must show that  $w(v) \neq h$ . If  $h(v) = h' < h$ , then by our induction assumption, we have that  $w(v) = h(v) = h' \neq h$ , and we are done. So assume that  $h(v) = h' > h$ . Then  $v$  has at least  $k$  neighbors  $u$  of height  $\geq h' - 1 > h - 1$ . By our induction assumption, for these  $u$  we have  $w(u) \geq h$ . Therefore,  $w(h') \geq h + 1$ . This establishes part (ii).  $\square$

### 3.3. Polynomial time $k$ -forward numbering: UpSlide

In this section we use the best features of Leap and Slide to give Algorithm UpSlide, a third s-s algorithm for finding a  $k$ -forward numbering. Algorithm UpSlide, shown in Figure 7, is similar to Algorithm Slide except that nodes only make *increasing* moves. The advantage is that it runs in time polynomial  $n$ , the size of the graph, and  $W$ , the largest initial value.

**Lemma 9** *Let  $G$  be a graph whose  $k$ -height is  $h$ . During Algorithm UpSlide, the values  $w(v)$  can never exceed  $W + h$ , where  $W$  is the largest initial value of  $w(v)$ .*

**Proof.** The proof is identical to that of Lemma 5.  $\square$

**Theorem 5** *If  $G$  has a  $k$ -forward numbering, then Algorithm UpSlide stabilizes in  $n(W + n)$  moves.*

**Proof.** By Lemma 4,  $G$  has a  $k$ -height numbering. Since the height of nodes are bounded by  $n$ , by Lemma 9, each node can move at most  $W + n$  times. Thus the sum of all moves is at most  $n(W + n)$ .  $\square$

**Theorem 6** *Algorithm UpSlide stabilizes if and only if the graph has a  $k$ -forward numbering. If it stabilizes, it produces a  $k$ -forward numbering.*

**Proof.** Clearly if Algorithm UpSlide has stabilized, then  $G$  has a  $k$ -forward numbering. The other direction follows from Theorem 5.  $\square$

We observe that the running time of Algorithm UpSlide indeed depends on both  $n$  and  $W$ , even when  $k = 2$ . Consider the path  $P_4$  on four nodes in which end nodes have been given a large integer  $W$  and internal nodes have been given zero and one. The internal nodes will alternately move, each time incrementing themselves by only two, and thus taking about  $W$  moves. Furthermore, we can show examples where Algorithm UpSlide takes  $\Omega(n(W + n))$ , so the bound in Theorem 5 is the best possible.

#### 4. Application: coloring

In this section we present a self-stabilizing (list) coloring algorithm that assumes a  $k$ -forward numbering is already given. In Section 4.2 we will drop this assumption.

##### 4.1. List coloring and the coloring number

Recall that the coloring number  $col(G)$  is the smallest number  $k$  such that  $G$  has a  $k$ -forward numbering. Its significance is that

$$\chi(G) \leq col(G) \leq \Delta(G) + 1, \quad (1)$$

which is a strengthening of Brooks' Theorem. The second inequality in (1) is obvious since any total ordering is a  $(\Delta(G) + 1)$ -forward ordering. To see the first inequality, it suffices to show that if  $G$  has a  $k$ -forward ordering, then  $G$  can be colored with  $k$  colors. Indeed, if  $v_1, \dots, v_n$  is a  $k$ -forward ordering, then color the nodes in descending order (i.e. beginning with  $v_n$ ), always choosing any available color, that is,

$$c(v_i) \in \{1, 2, \dots, k\} - \{c(v_j) \mid v_j \in N(v_i)\}.$$

Jensen and Toft [17] contains more discussion of the coloring number <sup>a</sup>.

Suppose that associated with each node  $v$  is a list  $L(v)$  of distinct colors. The minimum  $k$  such that, for any assignment of lists of  $k$  distinct colors, there always exists a coloring  $c$  on  $V$ , where  $c(v) \in L(v)$  for all  $v \in V$ , is called the *choice number* of the graph [17], and is denoted  $choice(G)$ . Clearly,  $\chi(G) \leq choice(G)$ . If we have a  $k$ -forward numbering  $(v_1, \dots, v_n)$ , from any list assignment of  $k$ -length lists, we can obtain a coloring by processing the vertices in reverse order,  $v_n, \dots, v_1$ , and always choosing any available color. Thus,  $choice(G) \leq col(G)$ .

Algorithm ColorByNumber, shown in Figure 8, assumes that each node  $v$  has a list  $L(v)$  of  $k$  colors, and a  $k$ -forward numbering  $w$  is known. We regard these as

---

<sup>a</sup>There  $col(G)$  is defined in an equivalent way, but in terms of  $k$ -back orderings.

|                                                                                                                                                                                                                                     |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>variable:</b> color $c(v)$<br><b>constant:</b> $k$ -forward numbering $w(v)$<br><b>constant:</b> list of $k$ distinct colors $L(v)$<br><b>recolor:</b> <b>if</b> $c(v) \in F_w(v)$<br>set $c(v)$ to any color in $L(v) - F_w(v)$ |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Fig. 8. Algorithm ColorByNumber

constants associated with each node. Note, however, that we obtain an ordinary  $k$ -coloring algorithm by defining, for each  $v$ ,  $L(v) = \{1, \dots, k\}$ .

For a node  $v$  and  $k$ -forward numbering  $w$ , define the set  $F_w(v)$  as those colors used by neighbors of  $v$  having numbers the same or larger than  $v$ 's:

$$F_w(v) = \{c(u) : u \in N(v) \wedge w(v) \leq w(u)\}.$$

The algorithm produces a  $k$ -coloring by giving the responsibility of rectifying monochromatic edges to the smaller-numbered node.

**Theorem 7** *Given any graph  $G$  and  $k$ -forward numbering, and any assignment of lists  $L(v)$  of size  $k$  to the vertices, Algorithm ColorByNumber stabilizes, producing a list coloring.*

**Proof.** First, note that the algorithm is well-defined in the sense that the set  $L(v) - F_w(v)$  is always nonempty: since  $w$  is a  $k$ -forward numbering, there is always a color that may be selected. Next, it is easy to see that the system is stable if and only if the values  $c(v)$  form a coloring. Finally, we claim that the system must stabilize. We will show that for any  $v$ ,  $m(v)$ , the number of moves made by  $v$ , is finite. For each node  $v$  we define

$$\begin{aligned} N_=(v) &= \{u \in N(v) \mid w(u) = w(v)\} \\ N_>(v) &= \{u \in N(v) \mid w(u) > w(v)\} \\ N_{\geq}(v) &= N_=(v) \cup N_>(v) \end{aligned}$$

Let us say that a move, made by a neighbor  $u$  of  $v$ , is *injurious* if  $c(u)$  becomes  $c(v)$ . Besides its initial move, a node  $v$  can make at most one move for every injurious move made by a member of  $N_{\geq}(v)$ . But moves made by members of  $N_=(v)$  are never injurious. Hence

$$m(v) \leq 1 + \sum_{u \in N_>(v)} m(u). \quad (2)$$

We now order the nodes  $v_1, v_2, \dots, v_n$  such that

$$w(v_1) \geq w(v_2) \geq \dots \geq w(v_n).$$

By induction on  $i$  we claim that each  $m(v_i)$  is finite. Clearly  $m(v_1) \leq 1$ . Inequality (2) completes the induction.  $\square$

In [14] and [16], s-s algorithms for  $(\Delta + 1)$ -colorings are given that run in  $n$  moves. Note that *any* numbering, say assigning all numbers  $w(v)$  to be equal, is a

|                                                                                                                                                                                                                                                                                                                                                          |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>variable:</b> nonnegative integer $w(v)$<br><b>variable:</b> color $c(v)$<br><b>constant:</b> list of colors $L(v)$<br><b>upslide:</b> if $v$ has at least $k$ larger or equal neighbors<br>set $w(v) = 1 + r_k(\{w(u) : u \in N(v)\})$<br><b>recolor:</b> if $c(v) \in F_w(v)$ and $ N_{\geq}(v)  < k$<br>set $c(v)$ to any color in $L(v) - F_w(v)$ |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Fig. 9. Algorithm k-Coloring

$(\Delta + 1)$ -forward numbering. In this case, the set  $F_w(v)$  is the full set of colors in  $N(v)$ . It is clear that in this case our algorithm runs in at most  $n$  moves. Therefore Algorithm ColorByNumber obtains a  $(\Delta + 1)$ -list-coloring, providing a generalization of the s-s algorithms in [14] and [16].

Algorithm ColorByNumber extends some of the results of Shukla et al. [19]. In particular, since a ring has a 3-forward numbering, Algorithm ColorByNumber can 3-list color rings without requiring an orientation, and since trees have 2-forward numberings, Algorithm ColorByNumber can 2-list-color trees.

#### 4.2. Coloring algorithm

By composing Algorithm ColorByNumber with any of the  $k$ -forward numbering algorithms we can obtain a general s-s algorithm for list  $k$ -coloring a graph  $G$ , which does not require an a-priori  $k$ -forward numbering. This algorithm is shown in Figure 9. As before, one obtains an ordinary  $k$ -coloring algorithm by taking the lists to be  $\{1, \dots, k\}$ . The algorithm stabilizes if and only if  $G$  has coloring number less than or equal to  $k$ . Our approach is modeled after Ghosh and Karaata [13], who obtain a self-stabilizing algorithm for 6-coloring planar graphs. The idea is to combine the two rules **upslide** and **recolor** from Algorithms UpSlide and ColorByNumber, respectively. Rule **upslide** is exactly the  $k$ -forward numbering rule from Algorithm UpSlide. Rule **recolor** is the list coloring rule of Algorithm ColorByNumber with an additional boolean constraint; a node must have less than  $k$  larger neighbors before it can change its color. This ensures that a new color is available.

**Lemma 10** *If  $G$  has a  $k$ -forward ordering, Algorithm k-Coloring makes at most  $nW + n^2$  upslide moves.*

**Proof.** The **upslide** moves use only variable  $w$  and this variable is not modified by the **recolor** moves. Therefore this is obvious in light of Theorem 5.  $\square$

**Lemma 11** *Algorithm k-Coloring can make only a finite number of consecutive **recolor** moves.*

**Proof.** Assume that there is an arbitrary numbering  $w$ , and we will consider a sequence of consecutive recoloring moves. Some nodes  $v$  can not move at all because  $|N_{\geq}(v)| \geq k$ . But in any case, after its first move, a node  $v$  can move at most once for every injurious move made by a member of  $N_{>}(v)$ . Thus an induction argument similar to that of Theorem 7 may be applied to show that  $m(v)$  is finite.

Order the nodes  $v_1, v_2, \dots, v_n$  such that

$$w(v_1) \geq w(v_2) \geq \dots \geq w(v_n).$$

Note  $v_1$  can move at most once, since it will never become injured by another node. By induction, let us assume that all of  $m(v_1), m(v_2), m(v_{i-1})$  are finite. To complete the induction we must show that  $m(v_i)$  is also finite. In addition to its first move, a node can only respond to injurious moves made by members of  $N_{>}(v_i)$ . That is,

$$m(v_i) \leq 1 + \sum_{u \in N_{>}(v_i)} m(u). \quad (3)$$

By our ordering,  $N_{>}(v) \subseteq \{v_1, \dots, v_{i-1}\}$ , and so by our induction hypothesis the right side of (3) is finite.  $\square$

**Theorem 8** *If  $G$  has a  $k$ -forward ordering, Algorithm k-Coloring stabilizes, and produces a  $k$ -coloring.*

**Proof.** It follows from Lemma 10 and Lemma 11 that Algorithm k-Coloring must stabilize. At this time, the values  $w(v)$  must form a  $k$ -forward numbering, or else some node could execute the **upslide** rule. The values  $c(v)$  must form a coloring, for otherwise, if there were a monochromatic edge  $vu$  with  $w(v) \leq w(u)$ , since the  $k$ -forward numbering is in place, node  $v$  could execute the **recolor** move.  $\square$

Any forest has a 2-forward numbering. It is also known [17] that series-parallel graphs have 3-forward numberings, and planar graphs 6-forward numberings. Therefore we have:

**Corollary 1** *Any forest is 2-colored by Algorithm k-Coloring.*

**Corollary 2** *Any series-parallel graph is 3-colored by Algorithm k-Coloring.*

**Corollary 3** *Any planar graph is 6-colored by Algorithm k-Coloring.*

**Corollary 4** *For any surface  $S$  there is a constant  $c_S$  such that all graphs embeddable on that surface are  $c_S$ -colored by Algorithm k-Coloring.*

**Proof.** The coloring number for all such graphs is bounded [17].  $\square$

Let  $G$  be a graph having maximum degree  $\Delta$ . Brooks' Theorem guarantees the existence of a  $(\Delta + 1)$ -coloring, and if  $G$  is not an odd cycle or a complete graph, there exists a  $\Delta$ -coloring. It is easy to show that graphs having maximum degree  $k = \Delta$ , that are not regular, must have a  $k$ -height numbering. Thus we have

**Corollary 5** *Any non-regular graph having maximum degree  $\Delta$  is  $\Delta$ -colored by Algorithm k-Coloring.*

## 5. Other models: distributed daemon and synchronous

In the distributed daemon model, the daemon may select an arbitrary subset of privileged nodes. The serial-model algorithms of this paper can be converted to s-s algorithms under the distributed daemon model at the cost of IDs and time.

In the synchronous model, all privileged nodes move simultaneously. All three of our  $k$ -forward numbering algorithms run correctly under this model. We know that at least one of these, Algorithm Slide, converges very fast.

**Theorem 9** *Under the synchronous model, Algorithm Slide stabilizes if and only if its graph  $G$  has a  $k$ -forward numbering. In this case, it stabilizes in at most  $n$  steps.*

**Proof.** Clearly if the algorithm stabilizes, then  $G$  has a  $k$ -forward numbering. On the other hand, assume that  $G$  has a  $k$ -forward numbering, and hence has a  $k$ -height numbering. To show that the algorithm converges in at most  $n$  steps, it suffices to show that nodes stop moving after  $h_k(G) + 1$  steps, where  $h_k(G)$  is the  $k$ -height of  $G$ . We claim that for each  $h' \geq 0$ ,

- (i) After  $h'$  steps, all nodes at height  $h'$  or larger, have values at least  $h'$ .
- (ii) After  $h' + 1$  steps, nodes at height  $h'$  have values exactly  $h'$ , which thereafter never change.

We prove this by induction. For  $h' = 0$ , both (i) and (ii) are obvious. Now let  $h > 0$ , assume that (i) and (ii) hold for all  $h' < h$ , and let  $v$  be a node having height  $h$ . Using an argument similar to that in Theorem 4, we let  $N(v) = A \cup B \cup C$ , where  $A$  contains the neighbors of height at least  $h$ ,  $B$  contains the neighbors of height  $h - 1$ , and  $C$  contains the remaining neighbors. We let  $a$  and  $b$  be the cardinalities of  $A$  and  $B$  respectively. It follows that  $a + b \geq k$ , for otherwise  $v$  would have smaller height. Also, by definition of height,  $a < k$ , and so  $b > 0$ .

Consider the step  $M_h$ . By this time,  $h - 1$  moves have taken place, so all nodes in the graph with height at least  $h - 1$  must have values at least  $h - 1$ . These include all members in  $A$  and  $B$ . It follows that  $x(v)$  is at least  $h$ . So after move  $M_h$ , node  $v$  has value at least  $h$ . This verifies condition (i).

Finally, consider step  $M_{h+1}$ : We also know that all members of  $B$  have stabilized having values exactly  $h - 1$ . It follows that  $x(v)$  is exactly  $h$ . This verifies condition (ii).  $\square$

One can also show that Algorithm Leap and Algorithm UpSlide also stabilize. Under the synchronous model, most algorithms need ID's in order to avoid infinite loops on symmetric graphs such as cycles. What is interesting here is that neither algorithm uses ID's. Part of the success is due to working with a restricted class of graphs. Thus, for example, one can find the center of a tree in the anonymous synchronous model.

Symmetry breaking is needed for the coloring algorithm. In the synchronous version of Algorithm ColorByNumber, one has to ensure that only the “smaller” node changes to rectify a monochromatic edge. So one defines the numbering of a graph to be the composition of the  $k$ -forward numbering and its ID (in other words, IDs are used for tie-breaking). The algorithm converges in linear time, in fact in  $d$  steps, where  $d$  is the diameter of the graph. Thus, for example, we have a synchronous  $(\Delta + 1)$ -list-coloring algorithm.

## 6. Dynamic programming algorithms in trees and $k$ -trees

In the case of trees, any 2-forward numbering provides a root-finding algorithm in that there is either a single node or pair of adjacent nodes which have the property

that no neighbor has a larger value. Thus algorithms such as **Leap** and **UpSlide** can be used to root a tree, at which time, any dynamic programming algorithm can be run. This improves the running time of the result in [12].

A  $k$ -tree is a graph which can be built recursively starting with a  $k$ -clique and then repeatedly choosing some  $(k - 1)$ -clique  $C_i$ , introducing a node  $v_i$  and joining  $v_i$  to all of  $C_i$ . Thus, a 2-tree is a normal tree, and a maximal outerplanar graph is a 3-tree.  $K$ -trees, and their subgraphs, the *partial  $k$ -trees*, have been well studied (see for example, [5] or [2]). It is clear that the coloring number of a  $k$ -tree is  $k$ .

Del Greco et al. [15] define the *kernel* of a  $k$ -tree as the nodes with maximum  $k$ -height. Unlike in a normal tree, there is no connection between the center and the kernel of a  $k$ -tree. It is, however, known that the kernel is either a  $k$ -clique or a  $(k - 1)$ -clique.

Using a technique similar to that in [12], a suitable dynamic programming algorithm for  $k$ -trees can be implemented by composing **Algorithm Slide** with the algorithm that calculates the dynamic programming equations at each node.

For example, Corneil and Keil [5] provide such an algorithm for the domination number of a partial  $k$ -tree. This algorithm uses an auxiliary structure called a *clique tree* (which is a tree) but this is reconstructible from the  $k$ -height numbering. We omit the details.

## Acknowledgements

The authors thank the National Science Foundation for NSF Grant **ANI-0218495** which helped support this research.

## References

1. G. Antonoiu and P.K. Srimani, "A self-stabilizing distributed algorithm to find the center of a tree graph," *Parallel Algorithms and Applications* **10** (1997) 237–248.
2. Stefan Arnborg and Andrzej Proskurowski, "Linear time algorithms for NP-hard problems restricted to partial  $k$ -trees," *Discrete Appl. Math.* **23** (1989) 11–24.
3. J.R.S. Blair and F. Manne, "Efficient self-stabilizing algorithms for tree networks," in *Proc. 23rd International Conference on Distributed Computing Systems, (ICDCS)*, Providence, Rhode Island, May 2003, pp. 20–27.
4. S.C. Bruell, S. Ghosh, M.H. Karaata and S.V. Pemmaraju, "Self-stabilizing algorithms for finding centers and medians of trees," *SIAM Journal on Computing* **29** (1999) 600–614.
5. D. G. Corneil and J. M. Keil, "A dynamic programming approach to the dominating set problem on  $k$ -trees," *SIAM J. Algebraic Discrete Methods* **8** (1987) 535–543.
6. M. Demirbas and H. Ferhatosmanoglu, "Peer-to-peer spatial queries in sensor networks," in *3rd IEEE International Conference on Peer-to-Peer Computing, (P2P '03)*, Linköping, Sweden, September 2003.
7. E. W. Dijkstra, "Self-stabilizing systems in spite of distributed control," *Communications of the ACM* **17** (1974) 643–644.
8. E. W. Dijkstra, "A belated proof of self-stabilization," *Distributed Computing* **1** (1986) 5–6.

9. S. Dolev and T. Herman, Superstabilizing protocols for dynamic distributed systems,” *Chicago Journal of Theoretical Computer Science* **3** (1997).
10. S. Dolev, *Self-stabilization*, (MIT Press, 2000).
11. S. Dolev, E. Schiller and J. Welch, “Random walk for self-stabilizing group communication in ad-hoc networks,” in *Proc. 21st IEEE Symposium on Reliable Distributed Systems*, 2002, pp. 70–79.
12. S. Ghosh, A. Gupta, M.H. Karaata and S.V. Pemmaraju, “Self-stabilizing dynamic programming algorithms on trees,” in *Proc. of the Second Workshop on Self-Stabilizing Systems*, 1995, pp. 11.1–11.15.
13. S. Ghosh and M.H. Karaata, “A self-stabilizing algorithm for coloring planar graphs,” *Distributed Computing* **7** (1993) 55–59.
14. M. Gradinariu and S. Tixeuil, “Self-stabilizing vertex coloration of arbitrary graphs,” in *4th International Conference On Principles Of Distributed Systems*, OPODIS’2000, Studia Informatica Universalis, 2000. pp. 55–70.
15. J.G. Del Greco, C.N. Sekharan and R. Sridhar, “Fast parallel reordering and isomorphism testing of  $k$ -trees,” *Algorithmica* **32** (2002) 61–72.
16. S. T. Hedetniemi, D. P. Jacobs and P.K. Srimani, “Linear-time self-stabilizing colorings,” *Information Processing Letters* **87** (2003) 251–255.
17. T.R. Jensen and B. Toft, *Graph Coloring Problems* (Wiley, 1994).
18. S. S. Kulkarni and U. Arumugam, “Collision-free communication in sensor networks,” in *Proc. Self-Stabilizing Systems: 6th International Symposium*, SSS 2003, San Francisco, (Springer-Verlag, Heidelberg, 2003) pp. 17 – 31.
19. S. Shukla, D. Rosenkrantz, and S. Ravi, “Developing self-stabilizing coloring algorithms via systematic randomization,” in *Proc. of the International Workshop on Parallel Processing*, Bangalore, India, 1994, (Tata-McGraw Hill, New Delhi) pp. 668–673.